

RandomX mining process

Start-to-finish audit of how an XMRig miner produces a Monero (XMR) share. Every step is the RandomX specification (tevador), which XMRig 6.26.0 implements. Monero consensus is RandomX v1. Integer arithmetic is modulo 2^{64} . This document is for reading the path, not a substitute for the C++.

Sources: tevador/RandomX doc/specs.md chapters 2, 4.6, 5, 7; XMRig src/crypto/randomx; portable pack XMR-Miner-Portable (-a rx/0 - -randomx-mode=fast). Official hash vector 1a is included so a reimplementaion can be checked.

0. What you are auditing

Mining is a lottery. The miner guesses a **nonce**, runs a fixed recipe, and gets a 256-bit number **R**. If R is smaller than a target T, the guess pays. The pool repeats the same recipe. Bytes must match. Close is rejected.

The recipe has two clocks. The slow clock builds a ~2 GiB read-only table (the dataset) from a chain key K, about every 2.8 days. The fast clock is one nonce: fill a 2 MiB notepad, run eight random programs, fold with Blake2b. Inside each program: 2,048 laps of the 13-step loop.

```
slow  K → Argon2d cache 256 MiB → dataset ~2,080 MiB    (~2.8 days)
fast  H = header || nonce
      S = Blake2b-512(H)
      P = AesGenerator1R(S)                                2,097,152 B notepad
      for program i = 0 .. 7:
        program VM from AesGenerator4R
        ic ← 2048
        while ic > 0:   steps 1-13      ← the inner loop
          S ← Blake2b-512(RegisterFile)
        A = AesHash1R(P)
      R = Blake2b-256(RegisterFile with A in last 64 B)
      valid iff interpret(R) < 2256 / difficulty
```

1. Get work (pool or node)

XMRig opens a TLS connection to the pool (portable pack: gulf.moneroocean.stream:20016, algorithm pinned -a rx/0). The pool sends a **job**:

Field	What it is
blob / header	Block template bytes. The nonce sits in a known offset of this blob. This is H once the nonce is written in.
seed / key	Hash of the RandomX key block. This is K. Changes when height % 2048 == 64, key block at height % 2048 == 0. Delay 64 blocks (~2 h).
target / difficulty	Share difficulty D. Win if $\text{interpret}(R) < 2^{256} / D$. Pool D is easier than network D; a share is not always a block.
job_id	Id so the miner can submit the right nonce against this blob.

Solo mining is the same recipe with network difficulty and a node instead of a pool. The hash function does not change.

2. Slow clock — build the 2 GiB book from K

Done once per key epoch, shared by all threads. Fast mode (- -randomx-mode=fast) keeps the whole table in RAM. Light mode keeps only the 256 MiB cache and recomputes items; same R, much slower. The portable miner uses fast mode.

2.1 Cache (256 MiB)

Argon2d, version 0x13, type 0 (data-dependent). Password = K (0–60 bytes). Salt = ASCII RandomX plus byte 0x03. Memory = 262,144 KiB blocks, 3 iterations, 1 lane. Argon2 finalizer is omitted; the filled memory is the cache.

2.2 Eight SuperscalarHash programs

BlakeGenerator seeded with K emits eight integer-only programs used only to expand cache → dataset. Not the same 256-op VM programs as the inner loop.

2.3 One dataset item (64 bytes)

Item number $n = 0, 1, \dots$ (~33.55 million). Registers $r0 \dots r7$ start as:

```
r0 = (n + 1) * 6364136223846793005
r1 = r0 XOR 9298411001130361340
r2 = r0 XOR 12065312585734608966
r3 = r0 XOR 9306329213124626780
r4 = r0 XOR 5281919268842080866
r5 = r0 XOR 10536153434571861004
r6 = r0 XOR 3398623926847679864
r7 = r0 XOR 9549104520008361294
```

Then eight rounds: load a 64-byte cache line, run SuperscalarHash[i] on $r0 \dots r7$, XOR the cache line into the registers, walk cacheIndex to another line. Concatenate $r0 \dots r7$ little-endian = one dataset item. XMRig does this at startup (--randomx-init=N threads). After that the book is read-only until K changes.

Audit check: you do not choose which item is read later. Step 7 names the address. Reading a different item yields a different R. Pool reject.

3. Fast clock — one nonce, one hash

XMRig writes a nonce into the blob. H = the blob as hashed. Then:

```
S      = Blake2b-512(H)                // 64 bytes
gen1   = AesGenerator1R(S)              // AES, fixed spec keys
P      = 2,097,152 bytes from gen1      // scratchpad / notepad
gen4   = AesGenerator4R(gen1.state)
fprc   = 0                             // roundTiesToEven
```

AesGenerator1R: 64-byte state, four 16-byte columns; each step one AES decrypt/encrypt round per column with keys = Blake2b-512("RandomX AesGenerator1R keys"). Output is the new state. Filling 2 MiB is 32,768 such steps.

Scratchpad levels (inclusive): L3 = whole 2 MiB, L2 = first 256 KiB, L1 = first 16 KiB. Access is address AND mask:

Level	8-byte mask	64-byte mask
L1 16 KiB	$(16384-1) \wedge \neg 7 = 0x3FF8$	—
L2 256 KiB	$(262144-1) \wedge \neg 7 = 0x3FFF8$	—
L3 2 MiB	$(2097152-1) \wedge \neg 7 = 0x1FFFF8$	$(2097152-1) \wedge \neg 63 = 0x1FFFC0$

Audit check: a different notepad fill (any generator except AesGenerator1R with those keys) is a different hash. Lattice/NTT/UHH fills are not this step.

4. Program the VM (once per program, eight times)

Need $128 + 8 \times 256 = 2,176$ bytes from AesGenerator4R (four AES rounds per column, two key sets from Blake2b-512 of "RandomX AesGenerator4R keys 0-3" and "... 4-7").

128 configuration bytes = 16 quadwords:

QW	Use
0–7	Initialize a0...a3 (read-only floats). Each half: fraction bits 0–51, exponent bits 59–63 (0–31). Value = $(1 + \text{frac}/2^{52}) \times 2^{\text{exp}}$, in $[1, 2^{32})$.
8, 10	ma, mx = low 32 bits. Always 64-byte aligned in use.
12 bits 0–3	Address registers: bit0→ r0 or r1; bit1→ r2 or r3; bit2→ r4 or r5; bit3→ r6 or r7. Named readReg0...3.
13	datasetOffset = (QW13 mod (EXTRA/64 + 1)) × 64. EXTRA = 33,554,368.
14–15	E-register masks: fraction bits 0–21, exponent bits 60–63. Used when converting notepad bytes to e0...e3.

Next 2,048 bytes copy into the program buffer: 256 instructions × 8 bytes. Then 4.6.1:

```
ic      ← 2048
spAddr0 ← mx
spAddr1 ← ma
r0...r7 ← 0
```

5. The 13-step loop (spec 4.6.2) — 2,048 times

This is the inner machine. Integer ops are modulo 2^{64} . A 64-byte XOR is eight little-endian u64s, r0 then r1 ... r7. v1: mp is an alias for mx. Monero uses v1.

Step 1 — mix notepad addresses

```
x      ← r[readReg0] XOR r[readReg1]
spAddr0 ← spAddr0 XOR low32(x)
spAddr1 ← spAddr1 XOR high32(x)
```

You do not pick these addresses. The last lap's registers do.

Step 2 — XOR L3 notepad into integers

```
addr ← spAddr0 AND 0x1FFFC0
r0...r7 ← r0...r7 XOR P[addr .. addr+63]
```

Step 3 — reload floats from L3 notepad

```
addr ← spAddr1 AND 0x1FFFC0
f0...f3 ← as_F(P[addr .. addr+31]) // each 8 B → two int32 → two exact doubles
e0...e3 ← as_E(P[addr+32 .. addr+63]) // same, then:
// sign = 0
// exponent bits 0-2 = 011
// exponent bits 3-6 = program E mask
// fraction bits 0-21 = program E mask
```

Step 4 — execute 256 instructions

Not one formula. 256 opcodes map onto 29 instructions (tables 5.2–5.5). Any 8-byte word is a valid instruction.

Frequencies:

Group	Ops / 256	Examples (all mod 2^{64} unless noted)
Integer 17 ops	120	IADD_RS: $\text{dst} \leftarrow \text{dst} + (\text{src} \ll \text{shift}) [+ \text{imm32} \text{ if } \text{dst}=\text{r5}]$; IMUL_R: $\text{dst} \leftarrow \text{dst} \times \text{src}$; IMULH: high 64 of 128-bit product; IXOR; IROR/IROL (count AND 63); INEG; ISWAP; IMUL_RCP: $\text{dst} \leftarrow \text{dst} \times \text{floor}(2^x / \text{imm32})$ with $\text{rcp} < 2^{64}$, nop if imm32 is 0 or 2^k .
Float 9 ops	94	FADD/FSUB vs group A; FMUL/FDIV/FSQRT on group E; FSCAL = XOR bits with 0x80F0000000000000; FSWAP halves. Rounding = fprc (IEEE 4 modes).
Control 2 ops	26	CFROUND: $\text{fprc} \leftarrow 2 \text{ bits of rotated src}$. CBRANCH: $\text{dst} \leftarrow \text{dst} + \text{cimm}$; jump if bits b..b+7 of dst are 0. Target = instruction after last modify of dst. Cannot infinite-loop.
Store 1 op	16	ISTORE: $\text{P}[\text{dst}+\text{imm32}] \leftarrow \text{src}$ (L1/L2/L3 by mod).

Memory ops: address = (src==dst ? 0 : r[src]) + imm32, then AND L1/L2/L3 mask. Read with src==dst → L3. Write with mod.cond ≥ 14 → L3. Else mod.mem==0 → L2, else L1.

Audit check: skipping “easy” instructions or running fewer than 256 is a different R. The eight-program chain exists so an easy program cannot be dropped either.

Step 5 — next DRAM address

```
mt ← ma
mx ← mx XOR low32( r[readReg2] XOR r[readReg3] )    // v1: mp ≡ mx
```

Step 6 — prefetch (does not change R)

```
prefetch D[ datasetOffset + (mx mod 2147483648) ]
```

Hardware hint for the next laps. Omitting it may slow the CPU; the hash is unchanged.

Step 7 — XOR 64 bytes from the dataset (the DRAM tax)

```
addr ← datasetOffset + (mt mod 2147483648)
r0...r7 ← r0...r7 XOR D[addr .. addr+63]
```

This is the forced random 64-byte RAM read. 8 programs × 2,048 laps = **16,384** such reads per hash. A “better” item is a wrong item. Pool reject.

Step 8 — swap memory registers

```
(mx, ma) ← (ma, mx)
```

So next lap’s load address is already in ma.

Step 9 — write integers to notepad

```
P[ spAddr1 AND 0x1FFFC0 ] ← r0...r7    // 64 bytes
```

Step 10 — mix F with E (RandomX v1)

```
f_i ← f_i XOR e_i    for i = 0,1,2,3
```

XOR of the 128-bit register representation, not a floating add. v2 uses AES here; Monero is v1. Using AES mix on current XMR is a different hash.

Step 11 — write floats to notepad

```
P[ spAddr0 AND 0x1FFFC0 ] ← f0...f3    // 64 bytes
```

Step 12 — clear notepad addresses

```
spAddr0 ← 0
spAddr1 ← 0
```

Step 13 — count down

```
ic ← ic - 1
if ic > 0: go to step 1
```

2,048 laps finish one program. Not the end of mining — see §6.

6. Eight programs, then fold

After `ic` hits 0 (except after the eighth program):

```
S          ← Blake2b-512(RegisterFile)    // 256 B: r0-r7 | f0-f3 | e0-e3 | a0-a3
gen4.state ← S
program VM from gen4                      // new 2176 bytes
ic ← 2048; r0..r7 ← 0; spAddr from new ma/mx
execute 4.6.2 again
```

Repeat until 8 programs have run. Last program skips the Blake2b reseed. Then:

```
A ← AesHash1R(P)                        // 64 B fingerprint of the 2 MiB notepad
                                         // notepad as AES round keys; two extra rounds
                                         // keys from Blake2b("RandomX AesHash1R state/xkeys")
RegisterFile[192:255] ← A               // overwrite a0-a3 slots
R ← Blake2b-256(RegisterFile)           // 32 bytes
```

Audit check: substituting UHH, a 64-bit mix, SHA-256, or a “compressed fingerprint” for AesHash1R or Blake2b-256 is a different R. Official vector 1a below is the check.

7. Lottery — is this nonce a share?

```
n = R interpreted as unsigned little-endian 256-bit integer
T = floor( 2^256 / D )           // D = difficulty
valid iff n < T
P(valid) = 1/D
E[hashes per find] = D
```

Same predicate as Bitcoin SHA-256d. RandomX only changed how R is born. A structured nonce (prime, 144000, golden angle) is still one ticket with probability 1/D. XMRig walks nonces sequentially / with thread stripes — more tickets per second, not luckier tickets.

Miss: `nonce++`, new H, new notepad, eight programs again. Dataset stays. Hit: submit to the pool (or broadcast a block if D is network difficulty).

8. Submit and verify

Miner sends `job_id`, nonce, and sometimes the hash. The pool runs RandomX on (K, H) and requires bit-identical R. Then it checks `n < T`. Accepted share → credited. Rejected → wrong bytes or stale job.

This is why an alternate construction only counts if every byte of R matches. The portable miner’s accepted/rejected counters are this check.

9. What XMRig actually runs (portable pack)

Flag / piece	Role in this document
xmrig 6.26.0 arm64	JIT of the VM (same R as the interpreter). Not a different hash.
-a rx/0	Monero RandomX v1.
--randomx-mode=fast	Full dataset in RAM (§2).
--randomx-init=N, --threads=N	Build the book with N threads; mine with N VMs, each with its own 2 MiB notepad.
-o gulf.moneroocean.stream:20016 --tls -k	§1 and §8.
caffeinate -i, ProcessType=Standard	Keep CPU awake; stay off efficiency-core parking. Not part of R.
donate-level=0	No hashrate diverted. Not part of R.

Work per hash: $8 \times 2,048 = 16,384$ DRAM reads, plus $8 \times 256 \times 2,048$ VM ops, plus AES fill/fold. On the source M2 this pack measured $\sim 3,212$ H/s at 8 threads — about 3,200 full recipes per second.

10. Official test vector (audit 1a)

From `tevador/RandomX src/tests/tests.cpp`, Hash test 1a, salt `RandomX\x03`, interpreter and JIT must agree:

```
K = "test key 000"
H = "This is a test"
R = 639183aae1bf4c9a35884cb46b09cad9175f04efd7684e7262a0ac1c2f0b4e3f
```

Further vectors (same file):

Id	K	H	R
1b	test key 000	Lorem ipsum dolor sit amet	300a0adb...9489c969
1c	test key 000	sed do eiusmod tempor...aliqua	c36d4ed4...2b77a8
1d	test key 001	sed do eiusmod tempor...aliqua	e9ff4503...7905fc

If a candidate implementation disagrees with 1a, it is not RandomX. XMRig’s self-test uses this family of vectors before mining.

11. Nested counts (one share attempt)

Layer	Count	What repeats
Dataset / K	1 per $\sim 2,048$ blocks	§2 book rebuild
Nonce / H	until $R < T$	§3–8
Programs	8 per nonce	§4 + §6
Laps (ic)	2,048 per program	§5 steps 1–13
Step 7 DRAM reads	16,384 per nonce	forced 64 B XOR
VM instructions	$8 \times 2,048 \times 256 = 4,194,304$ per nonce	step 4

12. Out of this path (for the audit)

These are real objects in other folders. They are **not** steps of rx/0:

Object	Why it is not RandomX
formula_uhh = 18 mod P	Not Blake2b. Official 1a starts 63...; UHH is 12 00 00...
nonce_generation.c (tetration)	Emits a uint64 walk, not R.
t2.py p-1 T_i	Prints prime-layer pairs. Never hashes (K,H).
HTML desk fold (64-bit mix)	Stub. Labeled as such. Fails vector 1a.
Angular / platonic nonce	Reorders tickets. P(win) stays 1/D.

Allowed “same result, different way”: another Blake2b, another AES, interpreter vs JIT. XMRig JIT is that. Same bytes as the spec interpreter.

13. End-to-end checklist

An auditor watching one share can tick:

#	Check
1	Job blob, K, D from pool. Algorithm rx/0.
2	Dataset resident ~2,080 MiB from this K (fast mode), or light-mode on-the-fly items that match spec 7.3.
3	Scratchpad = AesGenerator1R(Blake2b-512(H)), 2,097,152 bytes.
4	Eight programs from AesGenerator4R; r zeroed; ic=2048 each time.
5	Each lap: steps 1–13 as written. 16,384 dataset XORs. v1 F/E mix is XOR.
6	A = AesHash1R(P); R = Blake2b-256(register file with A in last 64 B).
7	$n = \text{LE256}(R)$; $n < 2^{256}/D$.
8	Pool recomputes R; bit-identical; then the inequality.

If any box fails, the share is not Monero RandomX. The portable miner is built to pass all eight.

Primary spec: <https://github.com/tevador/RandomX/blob/master/doc/specs.md> XMRig RandomX tree: <https://github.com/xmrig/xmrig/tree/master/src/crypto/randomx>